Limitations of Low-Code Platforms for a Complex Enterprise Use Case: A Case Study

Name: Madhumitha P

Roll No.: 241401049

Department: CSBS

A Case Study Report

241401049

Executive Summary

Low-code platforms speed up simple business apps, but they struggle when an enterprise system needs heavy logic, high volume, deep integration and strict compliance. This report shows that through an illustrative case: a mid-sized insurer, "Meridian Insurance", that tried to rebuild its claims-processing system on a leading low-code platform.

The first three months went well. Simple forms, approval flows and dashboards were ready in weeks instead of months. Then the real complexity arrived. Rules that depended on policy type, region and claim history became hard to model. Screens slowed down once the system handled thousands of claims a day. Connecting to a 20-year-old policy system needed custom code the platform could not host. Auditors asked for controls that the platform did not expose.

After 14 months, Meridian had spent about 70% of its budget, delivered about 40% of the scope, and decided to move the core engine to traditional code. Low-code stayed for forms and dashboards.

Main takeaway: low-code is a strong tool for the edges of an enterprise system, and a risky foundation for its core. The right question is not "low-code or code?" but "which parts of this system suit low-code?"

Note: Meridian Insurance is a composite case built from common patterns reported across industry experience. Names and figures are illustrative, not from a single real company.

1. Introduction

Low-code platforms let people build software by dragging, dropping and configuring instead of writing most of the code by hand. This report uses plain language to explain where that approach stops working for large enterprise systems.

What is low-code?

Think of low-code as building with ready-made blocks. The platform gives you pre-built screens, data tables, workflow steps and connectors. You arrange them visually and add small pieces of code only where needed. Well-known examples include OutSystems, Mendix, Microsoft Power Apps and Appian. Their cousin, "no-code", goes further and avoids code entirely, but with even less flexibility.

Why enterprises like it

- **Speed:** a first working version can appear in days or weeks.
- **Lower skill barrier:** business analysts, not only developers, can help build.
- **Fewer repeated tasks:** login, forms, hosting and basic security come built in.
- **Less backlog pressure:** IT teams can say yes to more small requests.

Why this matters

These benefits are real, so many companies try to stretch low-code from small tools to core systems. That is where problems start. A core enterprise system is not just a set of screens. It carries business rules, large data volumes, many connections to other systems, and legal duties. The more of these a system has, the less the "ready-made blocks" fit.

24140149

Purpose and scope of this report

This report has three goals:

1. Describe one realistic complex use case and how low-code performed on it.
2. Explain, in simple terms, which limits appeared and why.
3. Offer practical guidance on when to use low-code and when to choose another route.

The case is a composite. It is meant to teach patterns, so the numbers are round and illustrative.

2. The Case: Meridian Insurance

Background

Meridian Insurance is a regional insurer with about 3,000 employees and 1.2 million policyholders across four countries. Its claims system was built 20 years ago. It was slow, hard to change, and each new product took months to support. Leadership wanted a modern platform within 18 months.

After comparing options, the CIO chose a major low-code platform. The reasons were convincing on paper: faster delivery, a lower development bill, a visual way to involve business teams, and a vendor promise that it was "enterprise-grade".

What the new system had to do

The claims system is the heart of the business. It is not a simple app. The requirements fell into six groups:

Area	Requirement	Why it is demanding
Volume	15,000 new claims per day, 60,000 active at any time	Needs steady speed under load
Business rules	Over 900 rules that vary by product, country and claim history	Rules interact and change often
Integration	Link to 8 systems, including a 20-year-old policy system and external fraud and payment services	Old and new technology must talk to each other
Compliance	Audit trails, data residency by country, role-based access	Regulators can fine failures
Users	1,200 internal staff, plus partner garages and customers on web and mobile	Different needs per user group
Change	New rules and products every quarter	Must be changeable without breaking things

Team and plan

The program had 25 people: 10 developers (6 of them new to the platform), 6 business analysts, 4 testers, a product owner, an architect and 3 operations staff. The plan had three phases: (1) intake and approvals, (2) the claims engine with rules and payments, and (3) partner and customer portals.

Success measures

- Go live within 18 months, for 20% lower cost than a traditional build.
- Process a typical claim in under 2 seconds per screen action.
- Pass an external audit with no major findings.
- Cut the time to launch a new product from 6 months to 6 weeks.

3. Phase 1: Where Low-Code Worked Well

It is important to be fair: low-code delivered real value early. Understanding why it worked helps explain why it later stopped working.

Quick wins in the first four months

- **Claim intake forms:** a new claim form with validation and document upload was ready in 3 weeks. The old system would have needed about 4 months.
- **Approval workflows:** routing small claims to a supervisor, with reminders and escalation, was built mostly by business analysts using the visual workflow tool.
- **Dashboards:** managers saw live claim counts and ageing reports within days.
- **Notifications:** emails and text messages to customers were configured, not coded.

Why these were a good fit

All four tasks share the same traits. They are mostly about screens and simple flows. The data is small and clean. The logic is a few "if this, then that" steps. And no outside system is deeply involved. These are exactly the cases low-code was designed for.

Results at the end of Phase 1

Measure	Plan	Actual
Time to deliver Phase 1	6 months	4 months
Cost of Phase 1	$1.5M	$1.2M
User satisfaction (pilot team)	n/a	8 out of 10
Business analysts contributing directly	0	6

Leadership was delighted. The project was praised in the annual report, and the budget for Phase 2 was approved with little debate.

The hidden risk

Phase 1 was easy for a reason that nobody wrote down: it avoided the hard parts. Heavy rules, high volumes and legacy integration were all left for Phase 2. The early success created confidence that did not yet rest on evidence. This is a common trap, sometimes called the "demo effect": the easy 20% of the work gets done first and makes the whole project look easy.

4. Limitations Encountered (Part 1): Logic, Speed and Integration

Phase 2 started in month 5 and exposed the first three major limits. Each is explained below in simple terms, with what happened at Meridian.

Limitation 1: Complex business logic

The problem. Low-code tools model logic with visual flowcharts and rule tables. This is clear for 20 rules. It becomes unreadable at 900.

What happened. The claims rules depended on many things at once: policy type, country, claim amount, the customer's history, and fraud scores. One rule for "windscreen damage" alone had 14 branches. The visual flows grew into huge diagrams that nobody could read in full. Two analysts changed related rules in different places and created a conflict that sent some claims to the wrong approver for three weeks before anyone noticed.

Why it matters. Visual logic is hard to search, compare and review. In normal code, developers use functions, shared libraries and automated tests to keep complex rules tidy. Many platforms offer weaker versions of these tools.

Limitation 2: Performance and scalability

The problem. Low-code platforms add a layer between your design and the machine. That layer makes building easy but gives you less control over how fast things run.

What happened. In testing with 5,000 claims, everything felt fine. Under real load, with 60,000 active claims and 800 users at peak, the claim search screen took 9 seconds to open instead of the 2-second target. The cause was how the platform generated database queries behind the scenes. The team could not rewrite those queries directly. They could only reduce the data shown, add caches, and ask the vendor for help, which took weeks.

Why it matters. When you cannot reach the engine, you cannot tune it. Performance is often where the first serious pain appears, because it only shows up at full scale.

Limitation 3: Integration with legacy systems

The problem. Low-code platforms connect well to modern services with standard web interfaces (APIs). Old systems often speak older languages and formats.

What happened. The 20-year-old policy system exchanged data through fixed-width files and a message queue with a custom format. The platform had no ready connector. The team wrote a separate middleware service in Java and ran it outside the platform, then called it from the low-code app. The result was a hybrid that doubled the number of things to deploy, monitor and secure. The "no code" promise was quietly lost.

Why it matters. Large enterprises live with old systems for decades. A platform that cannot talk to them natively pushes the hard work back to traditional developers, often with more complexity than before.

How these three limits connect

All three come from the same cause: the platform makes decisions for you. That is a benefit when your needs are standard, and a cage when they are not.

5. Limitations Encountered (Part 2): Control, Compliance and Cost

As the project moved into months 8 to 14, a second group of limits appeared. These are less about technology and more about control, risk and money.

Limitation 4: Limited customization of screens and behavior

Partner garages needed a mobile screen that worked offline in repair shops with weak signal, and customers wanted a photo-based damage estimate. The platform's standard components could not do offline mode well, and the camera feature behaved differently on each phone. The team built custom components, but the platform's rules limited what they could change. Each workaround took longer than a normal build would have.

Limitation 5: Security and compliance

Auditors asked for three things: a full record of who changed which claim and when, data stored in each customer's own country, and encryption keys controlled by Meridian. The platform offered a basic audit log but not at the detail required. Data residency options covered only two of the four countries. Meridian could not inspect how the platform handled some security tasks, so it had to rely on the vendor's certificates and promises. Regulators, however, hold the insurer responsible, not the vendor.

Limitation 6: Vendor lock-in

Applications built on the platform could only run on that platform. The visual models and generated code could not be exported in a form that other tools could use. By month 12, Meridian had over 400 screens and flows tied to the vendor. Leaving would mean rebuilding almost everything. This gave the vendor strong negotiating power at renewal time.

Limitation 7: Cost that grows with success

Licensing started at $300,000 per year in the pilot. Pricing was based on users, apps and data volume. When the system grew to 1,200 internal users, plus partner and customer access, the quote at renewal rose to about $1.4M per year. Meridian had assumed that low-code meant low cost. In reality, the savings in development were partly offset by license fees that scaled with usage.

Limitation 8: Governance and maintainability

Because building was easy, many people built things. Analysts created small apps and automations, some of which duplicated each other. Nobody had a full map of what existed. When a platform upgrade changed how one component behaved, several apps broke in different ways. This is sometimes called "citizen developer sprawl": speed without structure becomes a maintenance burden.

Limitation 9: Testing and release practices

Professional teams rely on automated tests, version control and staged releases (development, testing, production). The platform supported these only in a limited way. Comparing two versions of a

2414010049

visual flow was awkward, and automated tests had to be built separately. As a result, the team did more manual testing, releases slowed down, and two defects reached production that a normal pipeline would likely have caught.

6. Analysis: Why These Limits Appear

The nine limitations look different, but they trace back to a few simple ideas.

Idea 1: Abstraction is a trade

Low-code hides detail so that building is easy. Hidden detail is also detail you cannot change. A car with automatic gears is easier to drive, but you cannot choose the gear when you need to. For ordinary roads this is fine. For a racetrack, you want the manual.

Idea 2: The 80/20 effect

Low-code handles the first 80% of a typical app very quickly. The last 20%, which holds the unusual cases, special rules and tricky connections, often needs custom work. In complex enterprises, that 20% is where most of the value and risk sit, and it can consume most of the time.

Idea 3: Enterprise systems are about more than screens

A claims system has to be fast at scale, auditable, secure, connected and changeable for many years. Low-code is strongest at the visible layer (screens and flows) and weakest in the invisible layers (data handling, performance tuning, security internals and release engineering).

Idea 4: Convenience creates dependency

The platform that saves time on day one owns your design on day 1,000. Lock-in and rising costs are not accidents. They are part of how the model works.

Summary table

Limitation	Root cause	Impact at Meridian	Severity
Complex business logic	Visual models do not scale to hundreds of linked rules	Rule conflicts, slow changes	High
Performance	Little control over generated queries and runtime	Search took 9s vs 2s target	High
Legacy integration	No native connector, custom middleware needed	Extra system to build and run	High
Customization	Fixed component library and platform rules	Offline mobile delayed 3 months	Medium
Security and compliance	Platform internals not open to inspection	Audit findings, residency gap	High
Vendor lock-in	Proprietary models, no clean export	Rebuild needed to leave	High
Cost growth	Pricing tied to users and volume	License rose about 4.7x	Medium

Limitation	Root cause	Impact at Meridian	Severity
Governance	Easy building without central oversight	Duplicate apps, upgrade breakage	Medium
Testing and release	Limited version control and test tooling	Slower releases, 2 defects in production	Medium

An important point on fairness

None of this means the platform was poor or the team was careless. The platform did what it was built to do. The failure was one of fit: a tool designed for fast, standard applications was asked to carry a system that is large, regulated and unusual. The mistake was made at selection time, when the use case was not tested against the platform's weak points.

7. Outcome and Impact

The turning point

In month 14, the steering committee reviewed progress. The numbers were hard to ignore.

Measure	Original plan	Position at month 14
Budget used	$9M total	$6.3M spent (70%)
Scope delivered	100% by month 18	About 40%
Typical screen speed	Under 2 seconds	4 to 9 seconds
Annual license	$300K assumed	$1.4M quoted at renewal
New-product launch time	6 weeks	5 months (rules too tangled)
Audit readiness	No major findings	5 open major findings

A simple projection showed that finishing on the same path would need another 18 months and about $8M more. The original business case, which promised lower cost, no longer held.

The decision

Meridian chose not to scrap the work. After a two-month review, the leadership team agreed on a hybrid approach:

- **Keep in low-code:** intake forms, approval workflows, dashboards, notifications and the partner portal. These worked well and were already live.
- **Move to traditional code:** the claims rules engine, the payment logic and the integration layer with the legacy policy system.
- **Add an API layer** between the two, so the low-code screens call the new core as a service.
- **Negotiate the license** down to the smaller footprint, with data export rights written into the contract.

Business impact

- **Time lost:** about 5 months spent on workarounds that were later discarded.
- **Money lost:** roughly $1.8M of work that had to be redone.
- **Morale:** the team felt frustrated, though the clear decision restored focus.
- **Reputation:** the CIO had to explain the change of direction to the board.

What went right

The loss was not total. The early forms and workflows stayed in use. Business analysts now understood the claims process more deeply than before. And because the review was honest and made at month 14, not month 30, Meridian avoided a much larger failure. Early courage to change course was the best decision in the whole program.

8. Alternatives and the Hybrid Approach

Meridian had more options than "all low-code" or "all custom code". The table compares the main routes in simple terms.

Option	Speed to start	Flexibility	Long-term cost	Best suited for
All low-code	Very fast	Low to medium	Rises with usage	Simple internal apps, prototypes
All custom code	Slow	Very high	High upfront, steadier later	Core systems with unique logic
Hybrid (low-code edges, coded core)	Fast to medium	High where needed	Balanced	Large enterprises with mixed needs
Buy a packaged product	Fast	Low	License plus customization	Standard processes (e.g. HR, finance)

How the hybrid model works

Think of the system in three layers:

1. **Experience layer (low-code):** screens, forms, portals and dashboards. These change often and benefit from speed.
2. **Core services layer (traditional code):** rules engine, calculations, payments, security and data. These need performance, testing and control.
3. **Integration layer (APIs):** a clear, documented set of connections between the layers and to outside systems. This keeps the platform replaceable.

The key design rule is that **business logic lives in the core, not in the screens**. If the vendor changes pricing or Meridian wants to switch platforms, only the experience layer must be rebuilt.

When low-code is a good choice

- The app is mainly forms, workflows or reports.
- Fewer than a few hundred users, or moderate data volumes.

- Logic is simple and stable.
- Few connections to other systems, and all are modern.
- Speed matters more than perfect fit, or the app may be short-lived.

When to be careful or choose another route

- Hundreds of interacting rules, or logic that changes weekly.
- High volume, tight response times, or heavy data processing.
- Strict regulation, audit and data-location needs.
- Deep connection to legacy or specialized systems.
- The system will be a long-term core asset of the company.
- Offline use, advanced graphics or very custom user experience.

A quick decision checklist

Before choosing a platform, ask these seven questions. Any "no" is a warning sign. If three or more items from the "be careful" list above also apply to your system, consider a hybrid or custom approach.

1. Can we test the hardest 20% of the use case first, not the easiest?
2. Can we see and tune performance at full production load?
3. Does the platform meet our audit and data-location rules today, not on a roadmap?
4. Can we export our work and leave if needed?
5. Have we modeled license cost at three times today's users?
6. Does it support version control, automated tests and staged releases?
7. Who will govern what gets built, and by whom?

9. Lessons Learned and Recommendations

Key lessons

1. **Test the hardest part first.** A pilot built on easy features proves little. Choose the most complex rule set, the heaviest data load and the trickiest integration, and build those during evaluation.
2. **Match the tool to the layer.** Low-code is a strong choice for screens and simple workflows. It is a weak choice for the engine room.
3. **Plan for the exit from day one.** Keep business logic in portable, documented services. Ask for export rights and data access in the contract.
4. **Count the full cost.** Include licenses at future scale, extra specialists, workarounds and the cost of leaving, not only the saving on development.

2414010049

5 **Do not mistake early speed for final speed.** Progress in the first months says little about month 12.

6 **Govern the builders.** Set rules on who may build what, how apps are named, reviewed, tested and retired.

7 **Be ready to change course.** Set review points with clear signals, and treat a change of direction as good management, not as failure.

Recommendations for enterprises

Stage	Recommended action
Before choosing	Write down the top 10 hardest requirements and run a proof of concept on them, with real data volumes
Choosing	Score vendors on performance tuning, compliance, export, testing tools and pricing at scale
Designing	Use a layered, hybrid architecture with business logic outside the platform
Building	Mix developers and analysts, with a central team that sets standards
Running	Monitor speed, cost and platform upgrades; keep an inventory of all apps
Reviewing	Hold a formal checkpoint every quarter against the original business case

Signals that trouble is starting

Teams that watch for early warning signs can act sooner. Typical signals are:

- Developers spend more time on workarounds than on features.
- Fixing one flow breaks another.
- Vendor support tickets about performance keep growing.
- A growing share of the work is custom code outside the platform.
- Cost forecasts keep being revised upward.

10. Conclusion

Low-code platforms are useful and here to stay. They let organizations deliver forms, workflows and dashboards faster, and they bring business people closer to software. Meridian's experience shows the value of that speed in Phase 1.

The same case shows the limits. When a system is complex, large, regulated and tied to old technology, the features that make low-code easy become obstacles: hidden internals, a fixed set of building blocks, and a dependency on the vendor. Logic becomes hard to manage, performance hard to tune, compliance hard to prove, and cost hard to predict.

The lesson is not to avoid low-code. It is to use it with open eyes: test the hardest parts first, keep core logic portable, and apply low-code where it fits best. A hybrid design lets an enterprise enjoy the speed of low-code at the edges while keeping control of the core. Organizations that choose this balance can move quickly without betting the business on a single platform.

24140104 9